Digital Twin

Telemetry Analyzer GUI with PySide6

Inhaltsverzeichnis

1. Project description

Aim

The **Digital Twin Analyzer** is a modular, GUI-driven interpretation platform designed to transform raw telemetry into real-time insight, diagnostics, and explainable analytics. As industrial systems increasingly rely on digital twins not only for simulation but for **monitoring, anomaly detection, and decision support**, the need for transparent, reproducible, and narratable analytical environments has become essential. The Analyzer was conceived to provide engineers, researchers, and consultants with a **real-time analytical cockpit** — a place where the behavior of a virtual machine can be explored, understood, and communicated with clarity.

At its core, the Analyzer unifies several complementary concepts: **incremental data ingestion**, **modular machine-learning pipelines**, **multi-tab visualization**, and **governance-ready health monitoring**. Where the Generator simulates behavior, the Analyzer interprets it. Statistical trends, clustering regimes, short-term forecasts, anomaly scores, keyword patterns, and explainable feature attributions all converge into a coherent analytical narrative. This transforms the Analyzer from a simple plotting tool into a **behavioral interpretation engine**, capable of revealing the rhythms, transitions, and hidden structures of a virtual drilling machine under continuous operation.

The system emphasizes **explainability, reproducibility, and operational transparency**. Every analysis session is grounded in the structured `config.json` produced by the Generator, ensuring that schema, sampling frequency, timestamp format, and alert configuration are interpreted consistently. The GUI foregrounds interpretability: users can observe time-series evolution, inspect clustering patterns, review anomaly spikes, read NLP summaries, and understand feature importances — all while monitoring system health, alerts, and logs. This transparency is essential in modern industrial analytics, where explainability and traceability are not optional but foundational.

A defining strength of the Analyzer is its **modularity**. Each subsystem — the incremental reader, analysis modules, visualization tabs, alert listener, health aggregator, and log panel — is designed to be independently replaceable and extensible. This modularity ensures that the platform can evolve as new analytical techniques, visualization modes, or telemetry formats emerge, while preserving a consistent architectural framework. Such adaptability is crucial in consulting and enterprise contexts, where analytical solutions must be tailored to diverse client environments without compromising reproducibility or governance.

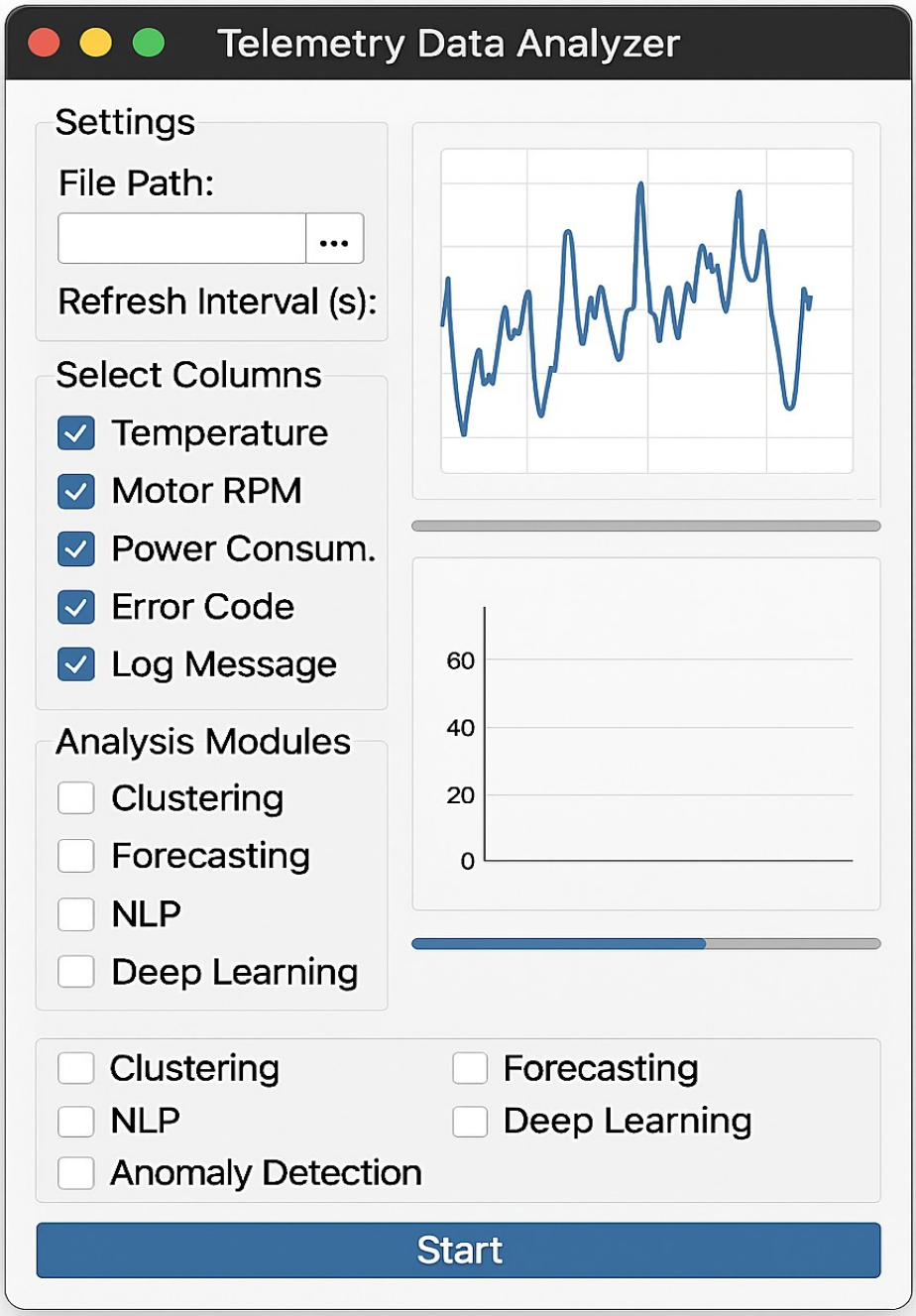
From a strategic perspective, the Digital Twin Analyzer demonstrates how **lightweight desktop applications can democratize access to real-time analytics and explainable AI**. By embedding streaming ingestion, modular ML pipelines, and multi-tab visualization into a PySide6 interface, the project makes advanced analytical concepts accessible to engineers, analysts, and non-technical stakeholders alike. Users can explore system behavior, inspect anomalies, and understand model attributions without writing code — fostering engagement, trust, and cross-functional collaboration.

🎯 Primary Aim

Industrial analytics systems must balance **real-time responsiveness**, **analytical depth**, and **interpretability**. The Analyzer achieves this by combining incremental data ingestion with modular ML pipelines, transparent visualizations, and a governance-ready health model. It provides a reproducible environment for exploring how telemetry evolves, how anomalies emerge, and how machine behavior can be explained in a way that is both technically rigorous and stakeholder-friendly.

In essence, the Digital Twin Analyzer is not merely a visualization tool — it is a **narrative platform for digital-twin analytics**. By transforming raw telemetry into transparent, reproducible, and communicable insights, it empowers teams to diagnose, validate, and explain machine behavior with confidence. The Analyzer bridges data ingestion, machine learning, and real-time visualization within a modern GUI, enabling users to interpret and document virtual machine behavior in a way that is both analytically powerful and intuitively accessible.

2. GUI and algorithmic concepts



The GUI sketch for the Digital Twin Telemetry Analyzer establishes a **clear, engineering-oriented workflow** that mirrors how real industrial telemetry systems are configured and monitored. The Analyzer GUI is structured around a **two-panel layout** with a bottom progress bar:

- **Left panel:**
  - telemetry file selection
  - refresh interval
  - row-limit settings
  - Start/Stop controls
  - module selection
- **Right panel:**
  - visualization tabs
  - system health summary
  - system log

This layout mirrors real industrial dashboards, where operators configure data ingestion on the left and observe system behavior on the right.

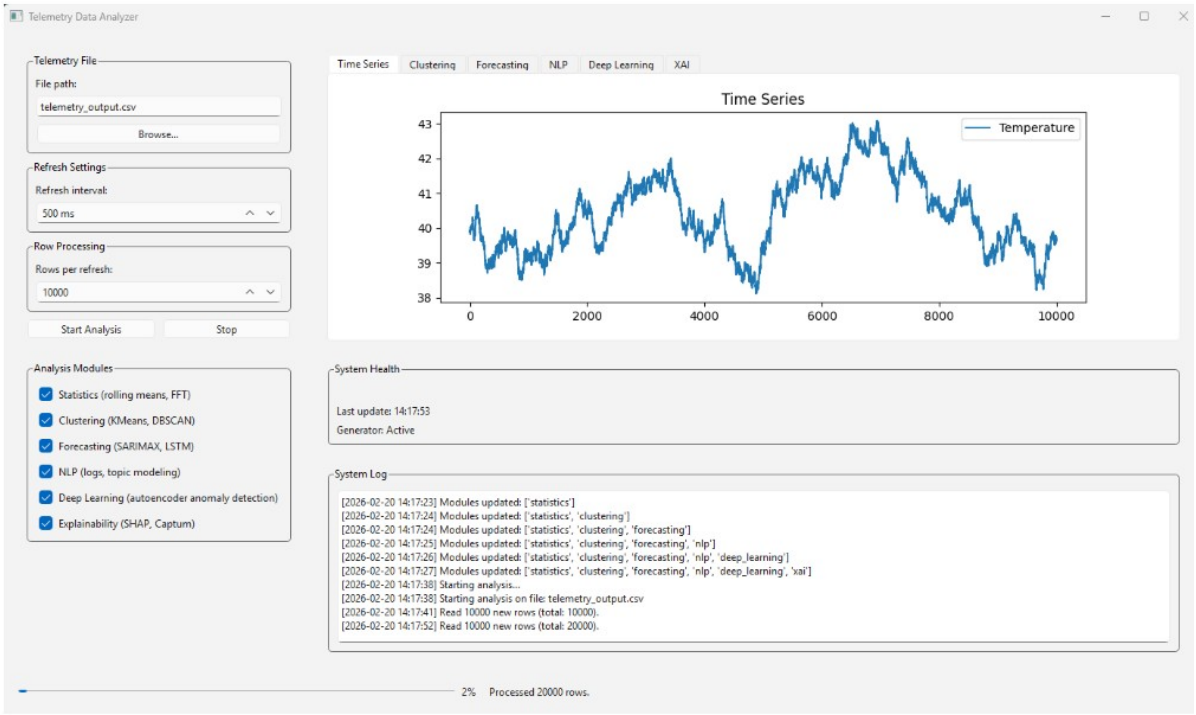
The Analyzer is the **interpretive half** of the digital twin pipeline:

- The Generator produces a **virtual machine**
- The Analyzer acts as the **virtual control center**

This separation mirrors real industrial architectures:

- **Edge layer:** data generation (sensors, PLCs, machines)
- **Cloud/Control layer:** analytics, dashboards, monitoring

By simulating both layers, your two-project suite provides a complete, pedagogical, and consulting-ready demonstration of digital twin principles.



The **Digital Twin Analyzer GUI** is structured as a professional, operator-grade dashboard — the kind you would expect in industrial control rooms, monitoring centers, or predictive maintenance platforms. Its layout is intentionally divided into **configuration**, **analysis**, and **diagnostics** zones, reflecting the workflow of real-time telemetry interpretation.

3. App design and its interaction flow

The GUI is organized into three conceptual layers:

4.1 Left Panel — Configuration & Module Selection (What should the Analyzer do?)

This panel defines the Analyzer's operational parameters:

Telemetry File Section

- File path (auto-loaded from `config.json`)
- Browse button for manual override
- Supports CSV and Parquet

This mirrors real industrial systems where analysts select a data source or switch between live and historical telemetry.

Refresh Settings

- Refresh interval (100–10,000 ms)
- Controls how often the Analyzer polls for new rows

This is analogous to a sampling rate for analytics rather than sensors.

Row Processing

- Number of rows per refresh (100–1,000,000)
- Defines batch size for incremental analysis

This allows the Analyzer to scale from lightweight demos to heavy telemetry streams.

Start / Stop Controls

- Start Analysis
- Stop

These buttons control the background analysis thread.

Analysis Modules

A set of checkboxes enabling or disabling:

- **Statistics** (rolling means, FFT)
- **Clustering** (KMeans, DBSCAN)
- **Forecasting** (linear regression)
- **NLP** (log keyword extraction)
- **Deep Learning** (IsolationForest anomaly scoring)
- **Explainability (XAI)** (RandomForest feature attribution)

This modular design mirrors real analytics pipelines where different models can be toggled depending on the use case.

4.2 Right Panel — Visualization, Health, and Logs (What is happening right now?)

This panel is the **operational heart** of the Analyzer.

Visualization Tabs

A tabbed interface with:

- **Time Series**
- **Clustering**
- **Forecasting**
- **NLP**
- **Deep Learning**
- **XAI**

Each tab displays results from the corresponding analysis module.
The plots update in real time as new telemetry arrives.

System Health

A compact health indicator showing:

- Current status (OK / Warning / Error)
- Last update timestamp
- Generator state (Active / Complete)

This panel integrates both:

- module-derived health signals
- alert messages from the Generator

System Log

A scrolling, timestamped console that records:

- module updates
- analysis start/stop events
- row ingestion events
- alerts from the Generator

This provides governance-ready traceability.

4.3 Bottom Bar — Progress Indicator

A progress bar shows:

- activity-based progress (0–100%)
- status messages ("Waiting for new data...", "Processed 200000 rows.")

Since the Analyzer does not know the total number of rows in advance, progress is based on activity rather than completion.

The above screenshots show the Analyzer in full operation, processing hundreds of thousands of rows in real time. Let's interpret what we see.

Time Series Tab

Shows temperature fluctuations over 10,000 samples.

Clustering Tab

Displays three distinct operational clusters.

Forecasting Tab

Shows a stable temperature trend with a short forecast horizon.

NLP Tab

Highlights frequent log keywords such as “low”, “minor”, “detected”.

Deep Learning Tab

Shows anomaly score spikes during high-load periods.

XAI Tab

Reveals Cycle Counter as the dominant feature (0.93).

System Health

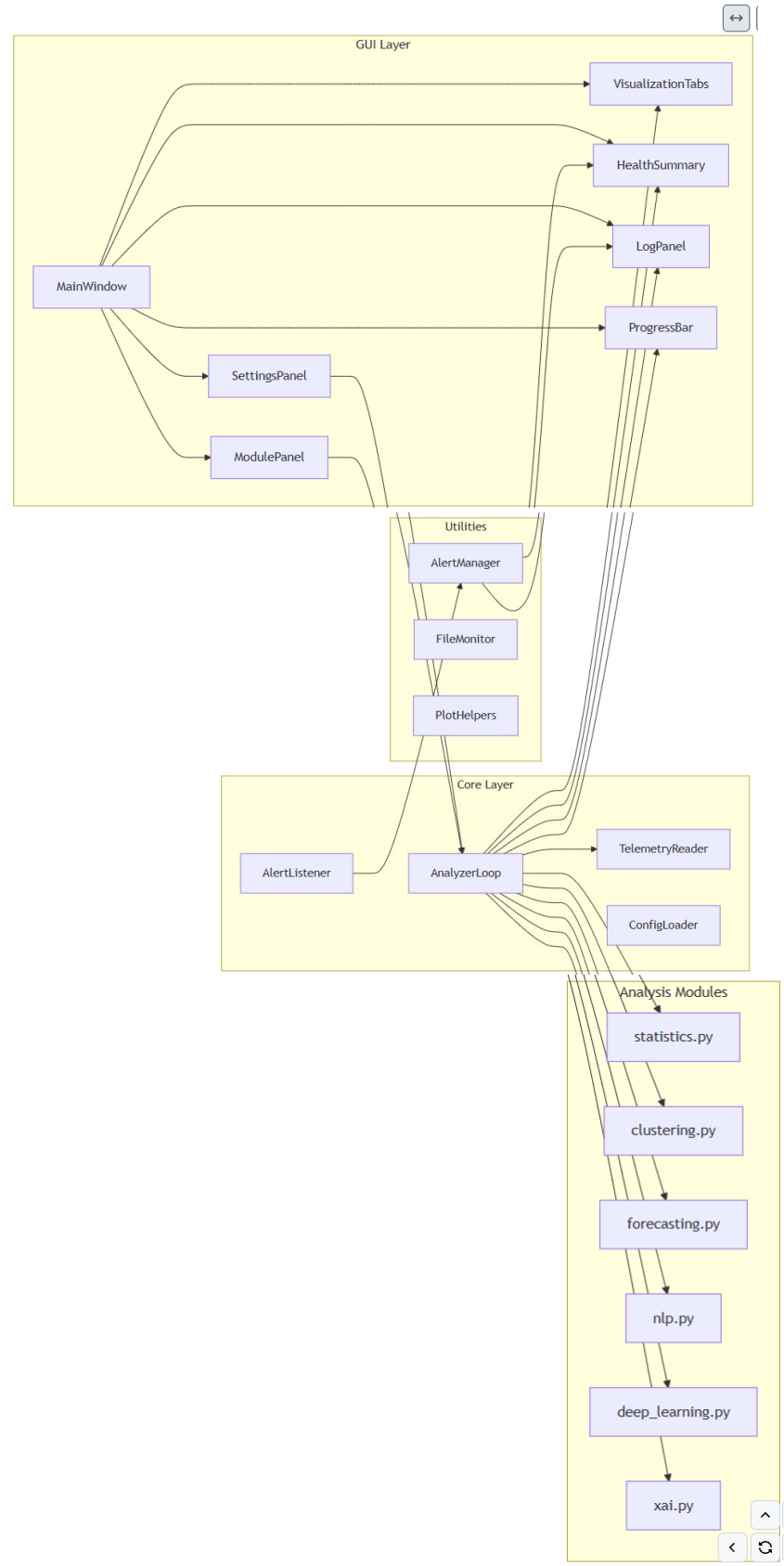
Displays warnings when attribution is unstable.

System Log

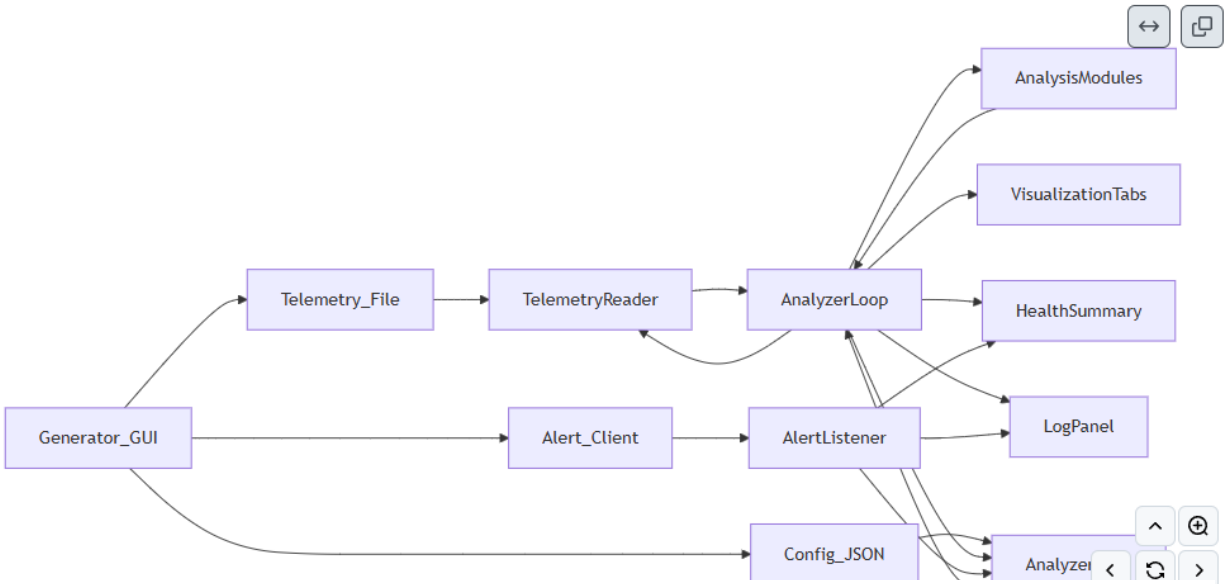
Shows incremental row ingestion up to 550,000 rows.

4. Run instructions and Mermaid diagrams

Module Interaction Diagram



6.2 Data Flow Diagram



6.3 Threading Diagram

11. Running the Generator from a Jupyter Notebook

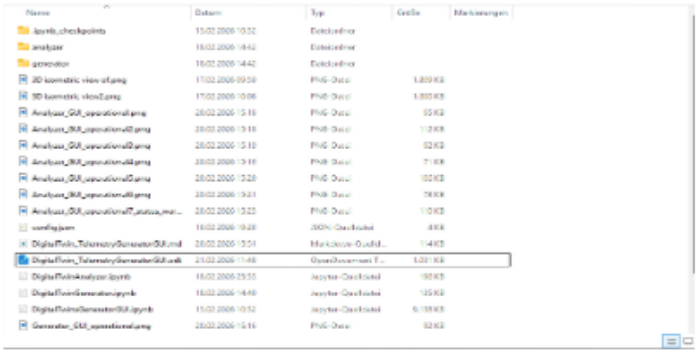
This section provides a clean, reproducible workflow for launching the Telemetry Generator from Jupyter.

✅ 11.1 Step 1: Download the folder and Environment Setup

Before running the generator from Jupyter, ensure the environment contains:

```
pip install PySide6 matplotlib numpy pandas pyarrow
```

Download the main folder
📁 DigitalTwinsGeneratorGUI
which has the following structure:



✅ 11.2 Step 2: Run the application

11.2.1 Launching the GUI from Jupyter

You can launch the full GUI directly from a notebook cell:

```
%run analyzer/app.py
```

This will:

- start the Qt event loop
- open the Telemetry Analyzer window
- allow full interactive use

The notebook cell will remain "busy" while the GUI is open — this is normal for Qt applications.

11.2.2 Running the Analyzer Programmatically (Headless Mode)

The Analyzer can be launched programmatically from a Jupyter notebook, enabling:

- automated testing
- batch analysis
- integration with the Generator notebook
- demonstration workflows

Basic Example

```
from analyzer.core.analyzer_loop import AnalyzerLoop
from analyzer.core.config_loader import load_config

config = load_config("config.json")

loop = AnalyzerLoop(
    config=config,
    progress_callback=lambda p, m: print(p, m),
    log_callback=print,
    visualization_callback=lambda r: print("Viz:", r.keys()),
    health_callback=lambda h: print("Health:", h),
)

loop.start(
    analysis_config={
        "file_path": config["output"]["file_path"],
        "refresh_interval_ms": 500,
        "row_limit": 10000,
    },
    selected_modules=["statistics", "clustering", "forecasting"]
)
```

This allows you to run the Analyzer headlessly for testing or integration.

11.3 ✅ Step 3: Interact with the Telemetry Analyzer

Interact with the Digital Twin Telemetry Analyzer by adjusting check boxes for **analysis module selection (statistics, clustering, forecasting, nlp, deep learning, etc)**, adjusting the **row processing parameters (refresh interval in ms, rows per refresh)**, and pressing the **Start Analysis** button.
Visual outputs of each selected numerical column update in real time within the Live Preview section of the GUI, and results can be saved as **data sets (Parquet/CSV)**.

5. Modularized Pythonic implementation

The Analyzer is built on a clean, layered architecture:

```
GUI Layer (PySide6)
│
├── SettingsPanel        → file path, refresh interval, row limit
├── ModulePanel          → module selection
├── VisualizationTabs    → plots and summaries
├── HealthSummary        → system health
└── LogPanel             → system logs
│
Core Layer
│
├── AnalyzerLoop         → background analysis engine
├── TelemetryReader      → incremental CSV/Parquet reader
├── AlertListener        → listens for Generator alerts
└── ConfigLoader         → loads shared config.json
│
Modules Layer
│
├── statistics.py        → rolling stats, FFT
├── clustering.py        → KMeans/DBSCAN
├── forecasting.py       → linear regression forecast
├── nlp.py               → keyword extraction
├── deep_learning.py     → anomaly scoring
└── xai.py               → feature attribution
│
Utilities
│
├── AlertManager         → formats and stores alerts
├── FileMonitor          → tracks file size and row count
└── PlotHelpers          → normalizes data for visualization
```

This separation ensures:

- GUI remains responsive
- analysis runs in background threads
- modules are independent and replaceable
- telemetry ingestion is efficient
- alerts integrate seamlessly

The full Pythonic code can be accessed via https://github.com/NenadBalaneskovic/ExternalProjects/blob/27acb5820635ba92eec30a6ee8db723bbe56dfba/DigitalTwinsGeneratorGUI/DigitalTwin_TelemetryAnalyzerGUI.md (sections 7 – 10 therein).

6. Results and conclusions

The Digital Twin Analyzer transforms raw telemetry into a structured, multi-layered analytical narrative. Each module contributes a different lens on the virtual drilling machine's behavior. Below is a unified interpretation of what the Analyzer reveals when operating on real-time telemetry streams.

12.1 Time Series Interpretation

The **Time Series** tab provides the most immediate view of the machine's state:

- **Temperature** curves reveal thermal drift, warm-up phases, and load-dependent heating.
- **RPM** curves show operational regimes, transitions, and stability.
- **Vibration** and **Noise** curves expose mechanical stress or imbalance.

A stable time series typically shows:

- smooth fluctuations
- bounded noise
- no sudden discontinuities

Anomalies appear as:

- spikes
- plateaus
- sudden drops
- oscillatory instabilities

This tab is the operator's "heartbeat monitor" for the digital twin.

12.2 Clustering Interpretation

The **Clustering** tab visualizes the machine's operational regimes in 2D feature space.

Typical patterns:

- **Three clusters** often correspond to *Idle*, *Low Load*, and *High Load*.
- **Outliers** may indicate transient anomalies or sensor glitches.
- **Cluster drift** over time can reveal wear or calibration issues.

If DBSCAN is used, a high noise ratio suggests:

- unstable sensor behavior
- inconsistent operating modes
- insufficient feature separation

Clustering is the Analyzer's way of discovering the machine's "behavioral vocabulary."

12.3 Forecasting Interpretation

The **Forecasting** tab overlays:

- historical data (blue)
- short-term predictions (orange)

This is useful for:

- early detection of overheating
- anticipating RPM overshoot
- predicting load-dependent drift

A stable forecast:

- follows the trend
- remains within realistic bounds
- does not explode or diverge

Warnings appear when:

- slope is unusually steep
- forecast values exceed physical limits

Forecasting provides a glimpse into the machine's *near future*.

12.4 NLP Interpretation

The **NLP** tab extracts patterns from log messages or categorical states.

Typical findings:

- frequent "idle", "low", "minor" → normal operation
- spikes in "error", "detected", "high" → anomalies
- repeated "no anomalies" → stable behavior

This module is especially useful when telemetry includes:

- operator messages
- system logs
- categorical states (e.g., Error Code, Operating Mode)

NLP gives the Analyzer a *linguistic* understanding of the machine.

12.5 Deep Learning Interpretation

The **Deep Learning** tab shows anomaly scores computed via IsolationForest.

Interpretation:

- **Low, stable scores** → normal behavior
- **Occasional spikes** → transient anomalies
- **Sustained high scores** → systemic issues

This module is sensitive to:

- vibration irregularities
- noise bursts
- multi-sensor inconsistencies

It provides a lightweight approximation of deep anomaly detection without heavy compute

12.6 XAI Interpretation

The **XAI** tab provides feature attribution for the primary numeric column.

Typical patterns:

- **Cycle Counter dominating** → strong correlation with drift
- **RPM contributing** → load-dependent behavior
- **Pressure/Load contributing** → mechanical stress

Warnings appear when:

- one feature dominates >0.9
- all importances are zero
- attribution is unstable

XAI provides transparency and governance-ready interpretability.

12.7 System Health Interpretation

The **System Health** panel integrates:

- module-derived health
- alert-derived health
- timestamp of last update
- generator state

g

Statuses:

- **OK** → stable operation
- **Warning** → unusual patterns detected
- **Error** → critical anomaly or module failure

This panel is the Analyzer's "mission control."

7. References

1. Tao, F., Qi, Q., Liu, A., & Kusiak, A. (2018). *Digital Twins and Cyber–Physical Systems in Manufacturing.* Engineering, 5(4); Grieves, M. (2015). *Digital Twin: Manufacturing Excellence through Virtual Factory Replication.*; Rasheed, A., San, O., & Kvamsdal, T. (2020). *Digital Twin: Values, Challenges and Enablers.* IEEE Access.; Challenges and Enablers. IEEE Access.; Microsoft. PySide6 Documentation.: https://pypi.org/project/PySide6/; Apache Arrow. Parquet File Format Specification.: https://arrow.apache.org/docs/python/parquet.html; NumPy Developers. NumPy Reference Guide.: https://numpy.org/doc/stable/reference/; Matplotlib Developers. Matplotlib Plotting Library.: https://matplotlib.org/;

2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).

3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).

4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.

5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).

6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).

7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).

8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).

9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).

10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).

11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).

12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).

13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).

14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).

15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).

16. Volker Ziemann: "Physics and Finance", Springer (2021).

17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).

18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).

19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).

20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).

21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).

22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).

23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).

24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).

25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).

26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).

27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).

28. David Foster: "Generative Deep Learning", O'Reilly(2023).

29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).

30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).

31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).

32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).

33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).

34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).

35. Kaggle-link: competition-documentation: https://www.kaggle.com/competitions/drw-crypto-market-prediction. R. Nystrom: „Game Programming Patterns“, 1st Ed. genever benning (2014); A. A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);

36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor’s Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);

37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).

38. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", https://arxiv.org/abs/2509.16242 (2025); D. C. Marinescu, G. M. Marinescu: "Classical and Quantum Information", 1sr Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024),

https://arxiv.org/pdf/2309.17368; Streamlit: https://streamlit.io/; Mitiq-package: https://quantum-journal.org/papers/q-2022-08-11-774/, https://arxiv.org/abs/2009.04417; Extrapolation packages: https://pypi.org/project/extrapolation/

39. A. Koop, H. Moock: "Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research", 1st Ed. Spektrum (2008); G, B, Dantzig, M. N. Thalpa: "***Linear Programming 1: Introduction***", 1st Ed. Springer (1997) & "Linear Programming 2: Theory and Extensions", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "Introductory Operations Research, Theory and Applications", 1st Ed. Springer (2004); D. G. Luenberger: "Linear and Nonlinear Programming", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "Operations Research - Introduction to Models and Methods", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "Modeling with Stochastic Programming", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "An Optimization Primer", 1st Ed. Springer (2021); cvxpy package: https://www.cvxpy.org/, https://pypi.org/project/cvxpy/; py-packages for operations research: https://wiki.python.org/moin/PythonForOperationsResearch

40. (Py-)tesseract package: https://github.com/tesseract-ocr/tesseract, https://pypi.org/project/pytesseract/, https://builtin.com/data-science/python-ocr, https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/ and UB Mannheim builds.

41. **Chip Huyen**, AI Engineering: Building Applications with Foundation Models, 1st Edition, O'Reilly Media, 2025; **Michael Lanham**, AI Agents in Action, 1st Edition, Manning Publications, 2025; **Melanie Mitchell**, Artificial Intelligence: A Guide for Thinking Humans, 1st Edition, Pelican Books, 2019; **Brian Christian & Tom Griffiths**, Algorithms to Live By: The Computer Science of Human Decisions, 1st Edition, Henry Holt and Company, 2016; **Ray Kurzweil**, The Singularity Is Nearer: When We Merge with AI, 1st Edition, Viking, 2024; OpenWeatherMap: https://openweathermap.org/, HuggingFace: https://huggingface.co/, LLM vs LRM: https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems, https://magazine.sebastianraschka.com/p/understanding-reasoning-llms

42. J. Frochte: "Finite-Elemente-Methode", Hanser 1st Ed.(2016); D. Gross, W. Hauger, J. Schröder: "Technische Mechanik 1-3", 15th Ed. Springer (2024); FEM-packages (Python): https://pypi.org/project/scikit-fem/, https://sfepy.org/doc-devel/index.html, https://getfem-examples.readthedocs.io/en/latest/demo_unit_disk.html, https://github.com/mlp6/fem. LLM vs LRM: https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems, https://magazine.sebastianraschka.com/p/understanding-reasoning-llms